

Общая схема работы с памятью в V8

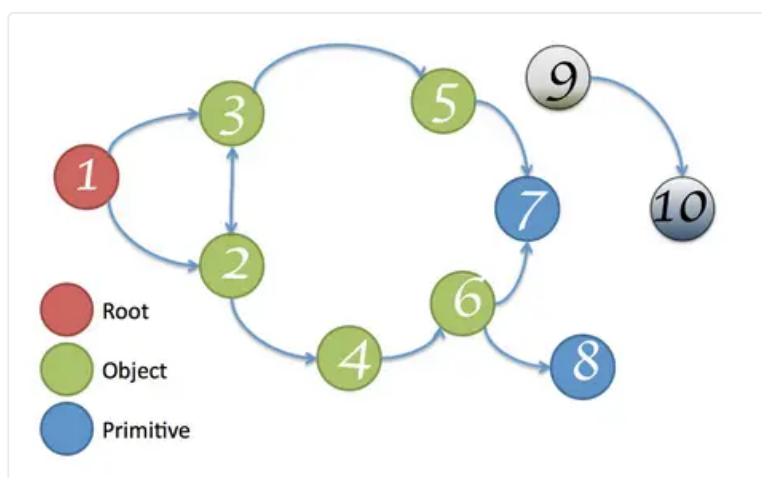
В браузере Chrome процессом компиляции и исполнения js-кода занимается написанный на C++ движок под названием V8. Подробнее про его работу описано [здесь](#).

Создаваемые объекты расположены в **куче** (js heap). Их можно представить в виде смешанного графа.

Вершины графа — это примитивы (строки, числа и т.д.) и объекты (массивы, DOM-элементы и т.д.). **Ребра** (дуги) направлены от родительского узла (тот, кто держит в памяти — e.g. массив) к дочернему узлу (тот, кого держат в памяти — e.g. элемент массива). Примитивы не имеют исходящих от себя ребер.

Граф имеет несколько **корней** (Root), но с точки зрения поиска утечек нас интересуют в первую очередь:

1. Глобальный объект (Window в браузере или Object в Node.js). К нему крепятся JS-объекты, а также обертки DOM-элементов, когда они взаимодействуют с JS-кодом.
2. Иногда объекты могут держаться за DevTools Console (если есть ошибка в ней или же вы что-то вычисляли в ней при отладке) или DevTools Debugger (отладка скрипта на вкладке Sources).



Для удаления неиспользуемых объектов из памяти имеется **сборщик мусора** (Garbage Collector или GC). Он проходит по ребрам кучи и смотрит, можно ли использовать вершину в дальнейшем, есть ли у нее связи с корнями через другие вершины или напрямую. Если таких связей нет, он помечает вершину мусором и очищает занимаемую им память (например, вершины 9-10).

Есть два режима работы GC — Minor и Major. Основное отличие в области кучи, по которой проходит сборщик: при Major он проходит по всей куче, а при Minor только по недавно созданным вершинам. Давно созданные вершины — это объекты, которые пережили некоторое количество минорных сборщиков мусора. Именно поэтому можно наблюдать такое явление, что после действий объем памяти сначала немного упал, и уже после истечения относительно большого промежутка времени (например, 10-20 сек.) упал на значительный объем.

Более подробно о сборщике мусора можете прочитать [здесь](#) и [здесь](#).

Таким образом, **утечкой** называется такой объект, который все еще имеет связь с каким-либо корнем, но при этом не может использоваться в дальнейшем. Значит, в теории, цикл для нахождения утечки следующий:

1. Создать объекты (т.е. выполнить тестируемое действие, например, открытие карточки).
2. Создать ситуацию, когда все созданные в п. 1 объекты не будут использоваться в дальнейшем (например, выполнить действие из UI, удаляющее построенные элементы — закрытие открытой карточки).
3. Вызвать или дождаться сборщика мусора.
4. Найти объекты, которые создались в п. 1 и остались в куче после п. 2 и п. 3.



Важно!

V8 для ускорения повторного исполнения скрипта при первом и втором его запуске создает некоторые объекты, которые сборщик мусора не удаляет. Также некоторые объекты могут подчищаться не при закрытии карточки, а при следующем ее открытии. Именно поэтому на практике цикл нахождения утечки необходимо прогонять минимум 3-4 раза, при этом отсекая первые 1-2 и последний повторы.

Куча обнуляется при перезагрузке страницы. Таким образом, после перезагрузки объем памяти всегда падает до первоначального. Это относится к Chrome, остальные могут не всегда подчищать память.